FuelPrint team plan

KIET InnoTech26 · Category 2, AI Solutions for Automation · for all five of us · version 2, 18 Oct 2026

What changed in version 2 (read this first)

- **FuelPrint is now a full AI and automation system.** Seven AI pieces and an automatic chain from the first sulphur rise to a sealed case. People sign only five decisions. Details below and in `contracts/AGREEMENT.md` sections 11 and 12.
- **Citizen SMS alerts.** Visitors sign up on their phone and get a normal SMS when smoke heads to their area and when the board decides.
- **Kiro now builds the shared core, the science, the AI, the alerts and the Field code** on Aditya Singh's Mac. Claude does the 3D look and reviews. Snowflake Cortex is no longer used.
- **The core is late by two days.** It lands on Mon 19 Oct, so every date moved. See "Dates".
- **New small jobs for you** are in your guide, marked **v2**. Abhivansh gets a new page: AI and automation (OP07).

What we are building

FuelPrint is an AI system that runs a pollution investigation by itself, from the first sulphur rise to a sealed evidence pack. People sign only the decisions that carry legal weight. It watches the air, ranks which factories could explain a rise, plans the next measurement, flies the drone, warns nearby people by SMS, keeps an honest evidence record, and learns from every closed case.

It runs as three web apps and a phone page on one shared server:

- **Operations** is the control room: sensor health, the investigation with the AI ranking, mission approval, the board decision, scenarios, and the AI and automation page.
- **Field** is the 3D world. The approved drone mission flies by itself (autopilot) or a visitor flies it from their phone. It samples the air at the real position of its inlet, and the sample is checked gas by gas.
- **Cases** is the evidence side: the screening record, a separate inspector specimen with seals and handovers, a lab result, a written finding, and a sealed evidence pack that anyone can verify.
- **Alerts** (`/alerts/`) is a phone page where visitors sign up for SMS alerts.

Everything is a simulation of a fictional industrial estate laid on the real Kundli map. We never name a real factory, never claim to detect a fuel, and never call anything legal proof.

This is the same FuelPrint that won IDEAVERSE. The early pitch said "find the factory burning coal at night". Our research showed that sensors cannot prove which fuel a factory burns, and that a box can miss a plume completely. So the product is now an investigation system that is honest about what it can't tell, and that honesty is what we pitch.

The AI, in one table

#	AI piece	What it does	Learns from
1	Smoke-spread reasoning	For each factory: "if this one were smoking, with this wind, what would the boxes read?" Factories that can't explain the readings drop out.	Nothing; it is physics maths, checked against real tracer experiments
2	Learned ranker	Puts the surviving factories in order. It can't bring back one that piece 1 dropped.	Thousands of simulated nights; version 2 trained on the KIET DGX A100
3	Planner	Picks the next measurement that best separates the suspects, within battery, time and safety	Nothing; it reasons fresh every time
4	Learning loop	Every closed case with a passing lab result becomes a lesson; the ranker retrains and must pass a quality check	Our own closed cases
5	AI wording	A chat model (Claude) rewrites explanations, findings and SMS in plain words. It can't change any number.	Nothing; an existing model
6	Normal-air model	Learns what normal SO2 looks like by hour and wind, and flags air that isn't normal	**Real** hourly SO2 from the CPCB station at Murthal, Sonipat
7	Autopilot	Flies the approved mission by itself when no visitor is flying	Nothing; it follows the approved route

The automation, in one chain

AUTO = the system does it by itself. HUMAN = a person decides.

1. AUTO: boxes report every second. The alarm fires. The normal-air model flags unusual air.
2. AUTO: the investigation opens, the AI ranks the suspects, the planner picks the next measurement.
3. AUTO: an SMS warning goes to opted-in phones in the downwind area.
4. AUTO: the mission is proposed. **HUMAN: the supervisor approves or rejects.**
5. AUTO: the approval expires by itself if the wind or the ranking changes.
6. AUTO: the autopilot flies (or a visitor does). Safety stops and the battery return are automatic.
7. AUTO: sampling runs by itself and a blocked tube is caught. **HUMAN: the QC person confirms each gas.**
8. AUTO: the ranking updates. The lab maths and QC are calculated.
9. **HUMAN: the case reviewer writes the finding.**
10. AUTO: the evidence is sealed and verified.
11. **HUMAN: the board decides.** AUTO: the board update SMS goes out.
12. AUTO: the learned ranker retrains from the closed case.

Every automatic step is a normal command sent by the server's `automation` actor, with a receipt, so every screen can show "done automatically". The presenter can switch each automatic step off to show the same run done by hand. Our line for judges: **"AI does the work, humans sign it."**

The five of us

Person	Group	Main job
Aditya Singh	Field	Owns the 3D world, drone, phone pages and SMS alerts. Runs Kiro, which builds the shared core, the science, the AI and the Field code. Merges everyone's work.
Aditya Singhal	Operations	Builds the Operations backend and most Operations screens with Codex

Person	Group	Main job
Abhivansh	Operations	Builds the sensor health panel, the hardware page and the AI and automation page. Main job: study the whole idea, the AI, the hardware and the judge questions. He is our pitcher.
Antara	Cases	Builds the Cases backend with Codex: cases, specimens, custody, lab maths, findings, the evidence vault
Aditya	Cases	Builds all Cases screens with Codex

To avoid mix-ups: "Aditya" alone in these docs always means the Aditya in Cases. The other two are always written with their surnames.

How the groups work at the same time

Normally the screens wait for the backend and the backend waits for the science. We avoid that in four ways.

1. One agreement. The file `contracts/AGREEMENT.md` in the repo fixes the exact shape of every message, command and record. Everyone builds against it. Only Aditya Singh changes it. If you think something is missing, ask in the group; don't change it yourself.

2. A fake server first. The shared server has a "replay" mode. It plays recorded test data through the same addresses the real server uses. Your screens work against it from the first day. When the real parts arrive you change nothing in your code.

3. Stubs for the science and the AI. Until the real ones are ready, the server uses stub versions that return fixed answers. Your code calls the same Java interfaces either way.

4. Own folders. Each person edits only their own folders (list below). Two people never edit the same file, so merges stay easy.

On **join day** we switch off the fake data, switch on the real science and AI, and fix mismatches together.

Who builds what

All coding is done through AI. You give Codex the prompts from your guide, check what it made, run the tests, and push.

Part	Built by	Runs on
Agreement, plans, reviews and merges	Claude	Aditya Singh's Mac
Shared core: server, database, live updates, clock, logins, automation actor, fake server, shared screen kit, start scripts	Kiro	Aditya Singh's Mac
Science engine: pollution simulation, sensors, alarm rule, smoke-spread reasoning, planner, scenarios, the night generator for training	Kiro	Aditya Singh's Mac
AI pieces: learned ranker, learning loop, normal-air model, AI wording, model training	Kiro (training of version 2 on the KIET DGX A100)	Aditya Singh's Mac, DGX
SMS alerts and the `/alerts/` phone page	Kiro	Aditya Singh's Mac, one Android phone
Field: drone, autopilot, phone joystick, capture and checks (code)	Kiro	Aditya Singh's Mac
Field: 3D world look	Claude	Aditya Singh's Mac
Operations backend and screens	Aditya Singhal with Codex	His laptop
Sensor health panel, hardware page, AI and automation page	Abhivansh with Codex	His laptop
Cases backend	Antara with Codex	Her laptop
Cases screens	Aditya with Codex	His laptop
Backup worker when Kiro hits limits	Codex	Aditya Singh's Mac

Languages and why

Layer	Choice	Why
Screens	React 18 + TypeScript, built with Vite	Fast to build, the AI tools know it well, type errors show up before the demo
3D	Three.js used directly	Full control over the scene and frame rate. Only Field uses it.
Charts	Recharts	Simple React charts for live readings
Backend	Java with Spring Boot 3.5, Maven wrapper	One language for the whole backend, easy for all of us to read
Database	SQLite through plain SQL (Spring JdbcTemplate, Flyway for tables)	Real SQL in one file. Nothing to install on stall laptops, and a backup is just a copy of that file.
Live updates	WebSocket (Spring)	The server pushes 10 updates a second to every screen and phone
Maths	Apache Commons Math	Matrices and least-squares fitting for the science engine
AI training	Python (LightGBM for version 1, PyTorch for version 2) in `tools/train/`	The standard tools for training models; version 2 uses the DGX A100's GPU
AI at run time	ONNX Runtime for Java	Runs the trained model inside our Java server, no Python at the stall
AI wording	Claude Haiku 5.5 through the Anthropic API	An existing chat model; if there is no internet, fixed text is used
SMS	"SMS Gateway for Android" app on one teammate's phone	Sends normal SMS from that phone's SIM; free and instant
Evidence	SHA-256 from Java's standard library; Ed25519 signing in the finale	Built into Java, no extra service

Java version: any JDK 17 or newer works (we compile for 17). Install Temurin 21 if you have nothing. Node 22 LTS for the screens. You do not need Python; only Aditya Singh's Mac trains models.

The repo

One private GitHub repo: **github.com/adityasinghin01-hash/fuelprint-innotech**. Your invite is in your GitHub email; accept it first.

```
contracts/          the agreement + reference numbers
Aditya Singh only
server/             one Spring Boot app (Java)
  .../fuelprint/core      server, database, live, clock
Kiro (Aditya Singh)
  .../fuelprint/api       interfaces between parts
Kiro (Aditya Singh)
  .../fuelprint/science   simulation, ranking, planner
Kiro (Aditya Singh)
  .../fuelprint/ai        learned ranker, normal air,
Kiro (Aditya Singh)
                          wording, learning loop
  .../fuelprint/alerts    SMS alerts
Kiro (Aditya Singh)
  .../fuelprint/ops       Operations backend
Aditya Singhal
  .../fuelprint/cases     Cases backend
Antara
  .../fuelprint/field     drone, autopilot, leases,
captures  Kiro (Aditya Singh)
apps/shared/        shared look, types, live connection
```

```
Kiro (Aditya Singh)
apps/operations/    Operations screens
Aditya Singhal, Abhivansh
apps/cases/         Cases screens
Aditya
apps/field/         3D world and phone joystick
Aditya Singh
apps/alerts/        SMS sign-up phone page
Aditya Singh
data/context/       Kundli map crop and wind (real data)
read only
docs/plan/          these documents
tools/              start scripts; tools/train = model
training
```

How to work with git

1. Each person works on their own branch: `ops/singhal`, `ops/abhivansh`, `cases/antara`, `cases/aditya`.
2. Commit small pieces often, with a clear message ("ops: approve mission screen").
3. Push and open a pull request into `main` at least once a day, by 9 PM.
4. Claude reviews every pull request on Aditya Singh's Mac. It merges if the tests pass and you only touched your own folders. Otherwise it comments with what to fix.
5. Every morning run `git pull origin main` and merge `main` into your branch before you start.

Never push straight to `main`. Never commit passwords, keys, phone numbers or `server/config/*.json`.

The agreement in short

The full file is `contracts/AGREEMENT.md`. These are the parts everyone must know.

- **The server decides everything.** Your screen shows what the server sends and asks it to do things. It never calculates a reading, a score, an AI output or an outcome itself.
- **A command gets a receipt.** To change anything, send `POST /api/commands` with a fresh `command_id`. The reply says APPLIED or REJECTED with a reason. If you retry, use the same `command_id`; the server will not do it twice.
- **Automatic steps are commands too.** The server sends them as the actor `automation`. Your receipt and event lists show `automatic: true` for them. Show "done automatically" on screen.
- **Five decisions are human only:** approve or reject a mission, review a capture, write a finding, record the board decision. Automation is refused for them.
- **Revisions stop overwrites.** Each command carries the `expected_revision` you last saw. If someone changed the record in between, you get `STALE_REVISION`. Reload and show the new state.
- **Live data comes from one socket.** `/ws/live` sends a `tick` 10 times a second (drone, readings, wind, clock) and an `event` whenever a record is saved. After a reconnect, ask for events after your last `cursor`.
- **Stale means read-only.** If nothing arrives for 3 seconds, show "stale" and disable buttons.
- **Unknown is null, never zero.**
- **Three outcomes are normal.** Supported, rejected and inconclusive all get a proper screen, not an error box.
- **Scores are not chances.** The ranking shows a fit number; the learned ranker shows a rank 1, 2, 3. Never write "87% guilty".
- **Hidden truth stays hidden.** No screen, API or AI text may know which factory is really emitting.
- **Phone numbers stay private.** You will only ever see masked numbers like +91 98xxxxxx78.

What we show at the department round

Five laptops, visitors' phones and one teammate's Android phone (the SMS sender), all on one Wi-Fi.

Laptop	App	Who stands there
1	Operations: health, investigation, approval	Aditya Singhal
2	Field: 3D world, drone and autopilot (also the host running the server)	Aditya Singh
3	Cases: lab check and lab result	Antara
4	Cases: inspector, custody, evidence vault	Aditya
5	Operations: board decision, AI and automation page	Abhivansh (also pitches)

The main story (about 7 minutes)

0. **Before it starts.** Visitors scan the alerts QR and sign up their phone for one area.
1. **Laptop 1.** The run starts on the fictional estate at night. Four sensor boxes report SO2, CO2 and particles every second. Box N2's SO2 climbs and stays up for 15 seconds, so the alarm fires and an investigation opens by itself. The normal-air model also marks N2 as unusual for this hour and wind.
2. **Phones.** Visitors signed up for East colony get an SMS: smoke may drift their way.
3. **Laptop 1.** The AI ranking shows which factories could explain it. Two still fit (S01 and S02), and the learned ranker puts them in order. The screen also shows what it can't tell, which readings it ignored, and the explanation in plain words.
4. **Laptop 1.** The planner picks the next measurement: the drone at a point downwind of both candidates at 35 m height, 30 seconds of valid air. It shows a cheaper option too. The mission is proposed automatically; the supervisor approves.
5. **Laptop 2 and a phone.** A visitor scans the QR code, gets a 6-letter code, and flies the drone. If nobody takes it within a minute, the autopilot flies it. The readings depend on where the drone's inlet really is. At the target, sampling runs by itself: wait, purge, settle, collect.
6. **Laptop 2.** Each gas is pre-checked automatically and the QC person confirms it: SO2 valid, CO2 valid.
7. **Laptop 1.** The ranking updates by itself. Now only S02 fits, inside this synthetic model.
8. **Laptops 3 and 4.** Cases receives the screening record. The inspector records a separate specimen with a seal, hands it over, the lab enters a result, and a finding is written with its limits. Legal suitability always says "not assessed".
9. **Laptop 4.** The case seals and verifies itself. "Tamper" changes one byte in a copy, and verify catches it.
10. **Laptop 5.** The board records a decision. An area-level notice goes out as an SMS to East colony, naming no factory.
11. **Laptop 5.** The AI and automation page shows the learned ranker retraining from this case, its test results including its misses, and the automation log of everything the system did by itself.

Two more stories that prove we are honest

- **Blocked inlet.** Same start, but the SO2 tube blocks during capture. The number on screen still looks normal, but the flow is zero. SO2 is rejected and CO2 is still usable. The case ends "rejected for use", and the learning loop refuses to learn from it.

- **Plume above the boxes.** A hot plume from a 45 m stack passes over every 3 m box. No alarm fires, no SMS goes out. The analyst schedules a survey, the drone result fits two explanations, and the case ends "inconclusive". A missed plume never becomes a "clean" verdict.

Judges can fly the drone, sign up for SMS, pick a scenario, block the inlet, switch automation off and on, and tamper with the evidence.

Dates

Date	What happens
Sun 18 Oct (tonight)	New docs. Accept the GitHub invite. Set up your laptop (last section). Read your guide and the agreement.
Mon 19 Oct, morning	Kiro puts the core foundation on `main` (server, database, commands). **Aditya Singhal and Antara start their backend steps.** Claude messages the group when it's there.
Mon 19 Oct, evening	The fake server and `@fp/shared` are on `main`. **Aditya and Abhivansh start their screens.**
Tue 20 Oct	Everyone builds their steps. Kiro builds the science, the AI pieces, the alerts and the Field code. Pull requests by 9 PM.
Wed 21 Oct	Morning: last steps. 2 PM: join (fake data off, real science and AI on, fix mismatches). Evening: full rehearsal on five laptops and phones. Freeze the department version at 11 PM.
22 to 24 Oct	Department round (slot not fixed yet). If our slot is the 23rd or 24th, join and rehearsal move one day later. Only bug fixes on the frozen version.
25 to 29 Oct	Finale work (list below), rehearsals, backup laptop drill.
Fri 30 Oct	Grand finale

What the finale adds

- **AI:** the learned ranker version 2, trained on the KIET DGX A100 on far more simulated nights; a test of the learned ranker against real tracer experiments; the normal-air model catching a weak plume the fixed alarm misses.
- More scenarios: wind turning mid-mission, two factories at once, a sensor fault, an unregistered source, a weak plume with background drift, combined errors.
- Operations: sensor placement and coverage view, teaching controls to change wind and stack height live.
- Field: instrument cutaway showing the pumps and gas paths, a lab workbench in 3D, the ground van, more polish.
- Cases: signed evidence (Ed25519), an export file, and a verifier that runs on a different laptop.
- Alerts: reply STOP by SMS to unsubscribe.
- Reliability: restart recovery, backup host, 60-minute rehearsal.
- Optional: the ₹2,000 bench sensor box, shown only as a relative dust reading, never as gas or fuel detection.

Things nobody says on stage

- "It detects coal." It can't. It shows a sulphur-bearing clue at most.
- "This factory is guilty." We show which registered sources fit the model, plus unknown sources.
- "87% probability" or "the AI is 90% accurate". Our scores are fit numbers and ranks. Model results are counts on test nights ("first in N of M test nights"), quoted only from the real test report, misses included.
- "The AI decides." The AI suggests; five decisions are always made by a person.
- "Our AI was trained on real factory data." No such data exists. The ranker learned from simulated nights; the normal-air model learned from real CPCB air data; the smoke maths is checked on real tracer experiments.
- "This is legal evidence." The inspector flow is a demo of the workflow. Legal suitability is always "not assessed".
- "No alarm, so the area is clean." A box can miss a plume completely.
- "These SMS are real warnings." Every SMS says "DEMO (simulated)".
- "Our hardware is tested." No physical test has been done yet. The hardware plan is a costed plan.
- "We have customers." We don't yet.

Set up your laptop today

1. Install **Git**. Check with `git --version`.
2. Install a **JDK 17 or newer** (Temurin 21 from adoptium.net if you have none). Check with `java -version`.
3. Install **Node 22 LTS** from nodejs.org. Check with `node -v`.
4. Make sure **Codex** runs: open a terminal and type `codex`. Sign in if asked.
5. **Accept the GitHub invite** in your email (or open github.com/adityasinghin01-hash/fuelprint-innotech/invitations).
6. Clone the repo: `git clone https://github.com/adityasinghin01-hash/fuelprint-innotech.git`
7. Create your branch: `git checkout -b ops/singhal` (use your own branch name from the list above).
8. Read your group guide, then `contracts/AGREEMENT.md`.

When the core is on `main` (Mon 19 Oct), these commands work:

Command	What it does
`tools/start-fake.sh`	Starts the server with recorded test data on port 8080
`tools/start-real.sh`	Starts the real server
`tools/dev-app.sh operations`	Runs one app with live reload on port 5173 (also `cases`, `field`, `alerts`)
`tools/test-all.sh`	Runs every Java and screen test
`tools/stall.sh`	Builds everything and starts the stall version on the host

On Windows use the `.ps1` versions of the same scripts.

When stuck: first paste the exact error into Codex and ask it to fix it. If that fails twice, message the group with the error text, what you tried, and the file name.